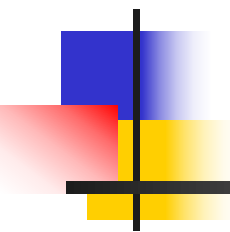


C语言程序设计

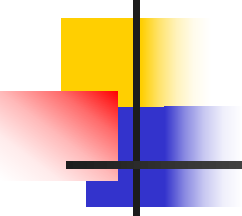
The C Programming Language



第9章 结构和联合

华中科技大学计算机学院

卢萍



主要内容

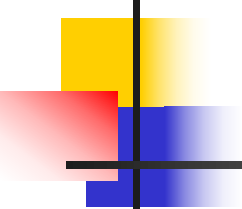
■ 结构类型

- 结构类型的声明和结构变量的定义
- 结构成员的引用（. 和->运算符）
- 结构数组和结构指针
- 结构做函数参数与返回值

■ 字段结构

■ 联合类型

■ 结构指针的应用：链表



结构类型

结构：一些相关联的数据的集合，这些数据的类型可以相同，也可以不同，这些数据都称为结构的成员。例如，

一个学生信息：姓名、成绩

```
struct stu {           /* 声明描述学生的结构类型 */
    char   name[20];    // 姓名
    float  score;       // 成绩
};
```

一颗行星的信息：名称、直径、卫星数

```
struct planet {        /* 声明描述行星的结构类型 */
    char    name[16];   /* 行星的名称 */
    double  diameter;   /* 行星直径的千米数 */
    int     moons;      /* 卫星数 */
};
```

例. 求平面内两点间的距离

```
#include<stdio.h>
#include<math.h>
struct point { /* 点结构类型的声明 */
    int    x;
    int    y;    /* x,y是点的坐标 */
};
int main(void)
{ struct point a, b; /* 声明点结构变量 */
  double dx,dy,d;
  printf( "输入两点的坐标: \n");
  scanf("%d%d%d%d",&a.x,&a.y, &b.x,&b.y);
  dx=b.x-a.x;
  dy=b.y-a.y;
  d=sqrt(dx*dx+dy*dy);          /*计算距离*/
  printf( "the distance is %f\n" ,d); /*输出*/
  return 0;
}
```

结构成员的访问：
结构变量名. 结构成员

将求距离定义为函数

```
double distance(struct point a1,struct point a2)
{
    double dx,dy,d;
    dx=a2.x-a1.x;
    dy=a2.y-a1.y;
    d=sqrt(dx*dx+dy*dy); /*计算距离*/
    return d;
}
```

结构类型的变量作为函数的参数
结构类型也可以作为函数的返回值

```
#include<stdio.h>
#include<math.h>
```

```
struct point { /*点结构类型的声明 */
    int    x;
    int    y;
};
```

```
double distance(struct point, struct point); // 函数原型
```

```
int main(void)
{
    struct point a, b; /* 声明点结构变量 */
    double dx,dy,d;
    printf( "输入两点的坐标: \n");
    scanf("%d%d%d%d",&a.x,&a.y, &b.x,&b.y);
    d= distance(a, b);          /*计算距离*/
    printf(" the distance is %f\n",d);    /*输出*/
    return 0;
}
```

将函数参数改为结构指针

```
double distance(struct point *p1, struct point *p2)
{
    double dx, dy, d;
    dx = (*p2).x - (*p1).x;
    dy = p2->y - p1->y;
    d = sqrt(dx*dx + dy*dy);
    return d;
}
```

结构成员的访问：
结构指针->结构成员

/*计算距离*/

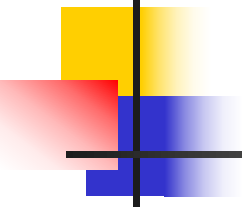
- 结构类型的指针作为函数的参数
- 结构类型的指针也可以作为函数的返回值

```
#include<stdio.h>
#include<math.h>
```

```
struct point{ /*点结构类型的声明 */
    int    x;
    int    y;
};
```

```
double distance(struct point *,struct point *); // 函数原型
```

```
int main(void)
{
    struct point a, b; /* 声明点结构变量 */
    double dx,dy,d;
    printf( "输入两点的坐标: \n");
    scanf("%d%d%d%d",&a.x,&a.y, &b.x,&b.y);
    d= distance(&a, &b); /*计算距离*/
    printf(" the distance is %f\n",d); /*输出*/
    return 0;
}
```

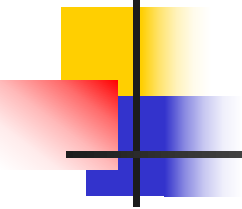



结构类型的变量作为函数参数

1、将实参结构拷贝给形参，占用内存较多，耗时间较长。

—— 适用于较小的结构

2、传值，形参结构不影响实参结构的值。



结构指针作为函数参数

1、将实参指针的值拷贝给形参指针。占用内存少，传参时间短。

—— 适用于较大的结构

2、传地址，在函数内部对形参指针的间访操作会影响实参指针所指结构变量的值。

—— 适合于要修改实参指针所指的结构变量值的情况。

结构变量的赋值操作

- **同类型**的结构变量可以相互赋值。例如：

```
static struct point {  
    int x;  
    int y;  
} a={1,2}, b;
```

则

b=a; /* 合法，对应的各个成员赋值 */

b={3,5} ; /* 错 */

b=(struct point){ 3,5}; /* 对，C99复合文字：结构常量 */



初始化点结构变量的函数

- 结构成员作为函数的参数
- 结构变量作为函数的返回值

```
struct point Point (int x, int y)
```

```
{
```

```
    struct point temp;
```

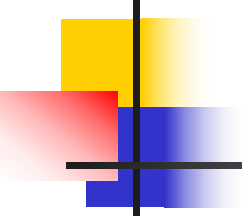
```
    temp.x = x;
```

```
    temp.y = y;
```

```
    return temp;
```

```
}
```

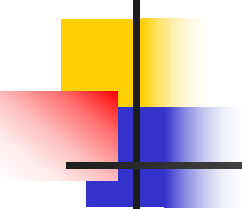
```
#include<stdio.h>
#include<math.h>
struct point { /*点结构类型的声明 */
    int    x;
    int    y;
};
struct point Point(int, int ) ;
double distance(struct point *,struct point *);
int main(void)
{
    struct point a, b; /* 声明点结构变量 */
    double d;
    a= Point(0, 0);
    b= Point(2, 3);
    d= distance(&a, &b);                /*计算距离*/
    printf( “the distance is %f\n” ,d);    /*输出*/
    return 0;
}
```



结构数组

设计一个能够描述学生成绩的结构类型，然后声明对应的结构数组，描述以下成绩表。

学号	姓名	性别	入学时间			计算机原理	英语	数学	音乐
			年	月	日				

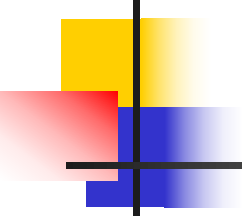


结构数组的定义

```
struct date
{
    int month;
    int damn;
    int year;
};
```

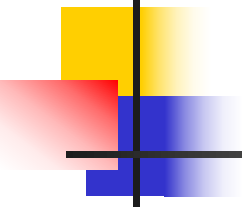
```
struct STUDENT
{
    int ID;
    char Name[10];
    char Sex[4];
    struct date timeOfEnter;
    int Computer;
    int English;
    int Math;
    int Music;
};
```

```
struct STUDENT stu[30];
```



结构数组的初始化

```
struct STUDENT stu[30] = {  
    {1, " Tom ", 'M', {1999, 12, 20}, 90, 83, 72, 82},  
    {2, " Nick ", 'M', {1999, 07, 06}, 78, 92, 88, 78},  
    {3, " Sue  ", 'F', {1999, 07, 06}, 89, 72, 98, 66},  
    {4, " May  ", 'M', {1999, 07, 06}, 78, 95, 87, 90}  
};
```

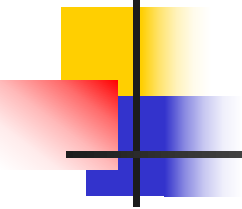
嵌套结构中结构成员的访问

结构变量名.结构成员名.成员名

理解为:

(结构变量名.结构成员名) .成员名

stu[0].timeofenter.year



用sizeof计算数组元素的个数

设有数组a，表达式：

sizeof a / sizeof a[0]

计算a数组中元素的个数。

计算表达式的值

设有：

```
char u[]="abcde";
```

```
char v[]="xyz";
```

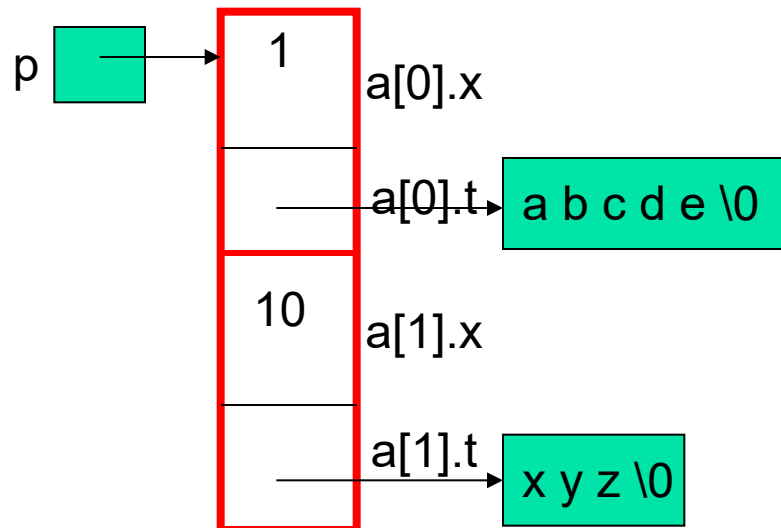
```
struct {
```

```
    int x;
```

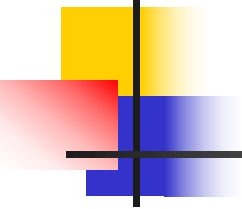
```
    char *t;
```

```
}a[ ]={{1,u},{10,v}}, *p=a;
```

各表达式相互无关。



表达式	值	表达式执行的操作
++p->x	2	访问x并使其增1
p->x++	1	访问x,取其原值参与运算,再使x增1
(++p)->x	10	p先自增,指向a[1],再访问x
p++->x , *p->t	'x'	逗号表达式,逗号右侧p指向a[1],访问t所指字符'x'
*p->t++	'a'	先访问t所指字符'a'后,t增1指向字符'b'
*++p->t	'b'	先访问t,然后t增1,再访问t所指字符'b'
*(++p)->t	'x'	p先自增,指向a[1],再访问t所指字符'x'



结构数组做函数参数

例：输入商品信息（包括商品编码、名称、价格），分别按照价格和名称排序，并输出所有商品信息。

商品编码	名称	价格（元）
1	笔	2.0
2	毛巾	10.5

结构类型声明和相关函数原型

```
#include<stdio.h>
```

```
#define N 10
```

```
typedef struct {
```

```
    long code;          /* 货物编码 */
```

```
    char name[20];      /* 名称 */
```

```
    float price;        /* 价格 */
```

```
} GOODS;
```

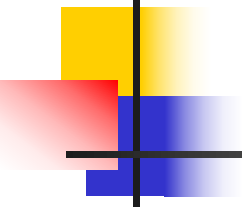
```
void input(GOODS *,int);    /*输入n件物品的信息 */
```

```
void display(GOODS *,int); /*显示n件物品的信息 */
```

```
void sort(GOODS *, int , int (*)(const void *,const void *)) /*排序 */
```

```
int cmpbyName(const void *,const void *); /* 按名称比较*/
```

```
int cmpbyPrice(const void *,const void *) ; /* 按单价比较*/
```



input函数

/* 输入n件物品的信息 */

void input(GOODS *p,int n)

{

int i;

for(i=0;i<n;i++){

scanf("%ld",&p[i].code);

scanf("%s",(p+i)->name);

scanf("%f",&(p+i)->price) ;

}

}

display函数

/*显示n件物品的信息 */

void display(GOODS *p,int n)

{

int i;

for(i=0;i<n;i++){

printf("%ld\t",(*(p+i)).code);

printf("%s\t",(p+i)->name);

printf("%f\n",p[i].price);

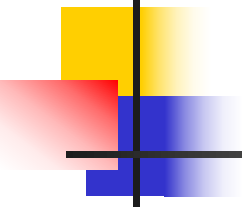
}

}

sort函数

/* 按函数指针fp指明的规则对n件物品排序 */

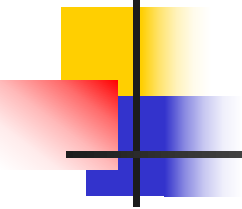
```
void sort(GOODS *p, int n,int (*fp)(const void *,const void *))  
{  
    int i,j;  
    GOODS t;  
    for(i=0;i<n-1;i++) {    // 冒泡法  
        for(j=i+1;j<n;j++) {  
            if(fp(p+i,p+j)) {  
                t=*(p+i);    // *(p+i)等价于p[i]  
                *(p+i)=*(p+j); // *(p+j)等价于p[j]  
                *(p+j)=t;  
            }  
        }  
    }  
}
```

回调函数：cmpbyPrice

/* 按单价比较*/

```
int cmpbyPrice(const void *s,const void *t)  
{  
    const GOODS *p1,*p2;  
    p1=(const GOODS *)s  
    p2=(const GOODS *)t;  
    if(p1->price < p2->price) // 按价格降序  
        return 1;  
    else return 0;  
}
```

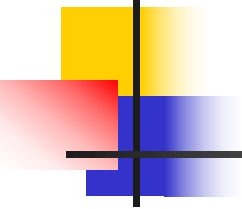


回调函数：cmpbyName

/* 按名称比较 */

```
int cmpbyName(const void *s,const void *t)  
{  
    const GOODS *p1,*p2;  
    p1=(const GOODS *)s;  
    p2=(const GOODS *)t;  
    return(strcmp(p1->name,p2->name)>0);  
}    // 按名称升序
```

```
int main()
{
    GOODS g[N];
    input(g,N);
    display(g,N);
    sort(g,N, cmpbyName);
    display(g,N);
    sort(g,N, cmpbyPrice);
    display(g,N);
    return 0;
}
```



9.6 联合

- 与结构类似，联合类型也是一种构造类型。
- **结构变量**是占据各自不同空间的各成员变量的集合。
- **联合变量**是占用同一内存空间的各成员变量的集合。
- **联合特点：各成员共享存储**

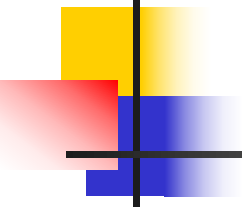
联合变量的定义和初始化

- 除了用关键字**union**取代**struct**之外，联合类型的定义、联合变量的声明、以及联合成员的引用在语法上与结构完全相同。但是**只能对联合的第1个成员进行初始化。**

```
union chl {  
    char c;  
    short h;  
    long l;  
} v = {'9'};
```

- 联合所有成员都是从同一存储空间的边界（低地址）开始存放，所以**联合各成员的地址和联合变量的地址是相同的**，但是**类型不同**。

&u	&u.c	&u.h	&u.l	值相同
union chl *	char *	short *	long *	



分离short数据的高低字节

```
union {  
    short n;    //存放要进行分离的数据  
    char a[2]; //n与数组a占的字节数相同  
} test;  
  
test.n=0x1234;  
low =test.a[0]; //低字节数据, low=0x34  
high=test.a[1]; //高字节数据, high =0x12
```

判断大小端

如何检查处理器是big-endian还是little-endian?

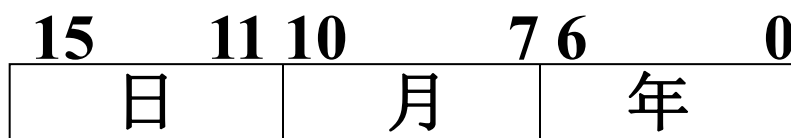
联合体union的存放顺序是所有成员都从低地址开始存放，利用该特性就可以轻松地获得了CPU对内存采用Little-endian还是Big-endian模式读写。

```
/*return 1 : little-endian, return 0:big-endian*/  
int checkCPUendian( )  
{ union {  
    unsigned int a;  
    unsigned char b;  
} c;  
  c.a = 1;  
  return (c.b == 1);  
}
```

9.7 字段结构

压缩：把表示21世纪日期的日、月和年3个整数压缩成1个16位的整数。

分析：因为日有31个值，月有12个值，年有100个值，所以可以在一个整数中用5b表示日，用4b表示月，用7b表示年。



字段：字中一组相邻的二进制位。是指定了存储位数，类型为**unsigned int**（或**unsigned short**）的结构成员。

字段结构在操作系统，编译程序，计算机接口的C语言编程方面使用较多。

用字段结构实现

字段结构的声明:

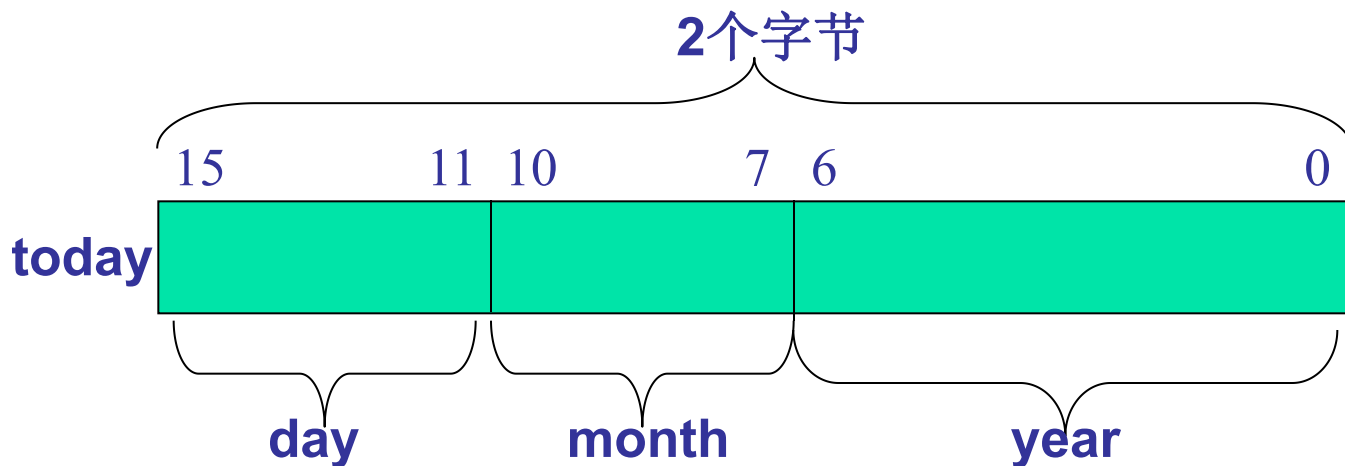
```
struct date {  
    unsigned short year : 7 ;    /* 年 */  
    unsigned short month : 4 ;   /* 月 */  
    unsigned short day : 5 ;     /* 日 */  
};
```

↑ ↑
字段名 字段宽度

字段的宽度: 组成字段的二进制位的数目, 是一个非负的整型常量表达式。

字段结构变量

```
struct date {  
    unsigned short year : 7 ;    /* 年 */  
    unsigned short month : 4 ;   /* 月 */  
    unsigned short day : 5 ;     /* 日 */  
};  
struct date today; /* today是date 字段结构变量 */
```



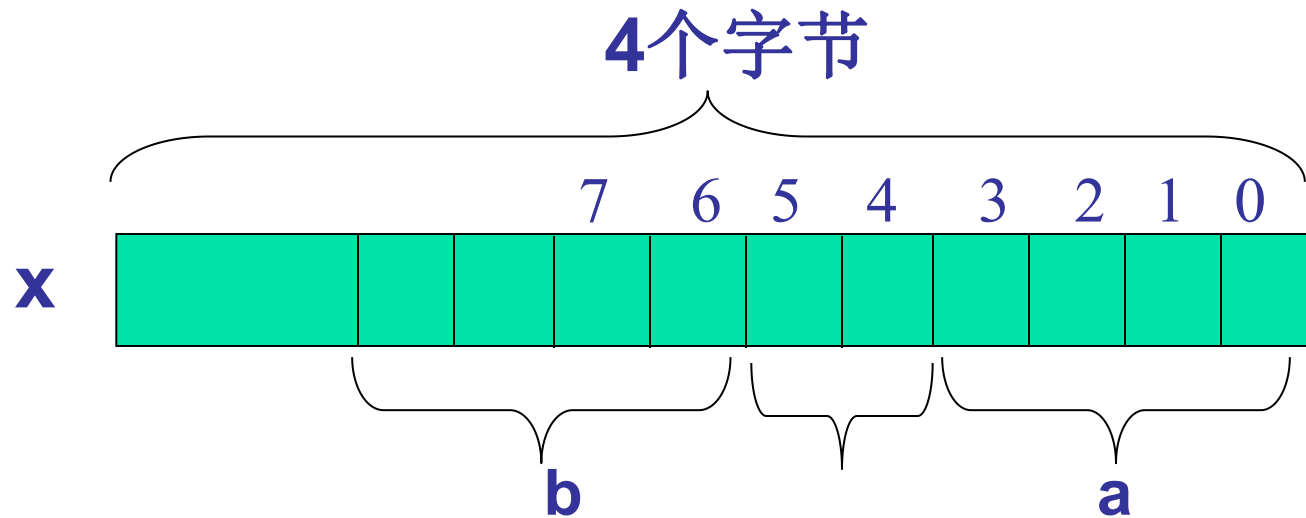
引用字段

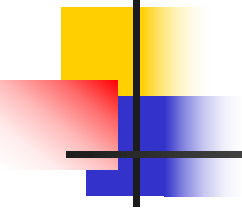
- ◆ 与结构完全相同： . 或 ->
- ◆ 字段就是一个小整数，它可以出现在其它整数可以出现的任何地方。字段在参与运算时被自动转换为 int 或 unsigned int 类型的整数。

```
today.year=13;    (2013)
today.month=5;
today.day=9;
```

无名字段（匿名）

```
struct {  
    unsigned a: 4;  
            : 2;  
    unsigned b: 4;  
} x;
```





字段结构与联合的应用

如何访问**16位字**中的高低字节和各二进制位？

- ◆ 定义**8位宽**的字段**byte0**、**byte1**
 - 表示一个**16位字**中的高/低字节
- ◆ 定义**1位宽**的字段**b0~b15**
 - 表示一个**16位字**中的**bit**
- ◆ 定义联合类型
 - 使一个**short**变量、**2个byte**字段与**16个bit**字段共享存储。

字段结构与联合的应用

```
#include<stdio.h>
```

```
struct w16_bytes{
```

```
    unsigned short byte0:8;    /* byte0: 低字节*/
```

```
    unsigned short byte1:8;    /* byte1: 高字节 */
```

```
};
```

```
struct w16_bits{
```

```
    unsigned short b0:1,b1:1,b2:1,b3:1,b4:1,b5:1,b6:1,b7:1,  
                  b8:1,b9:1,b10:1,b11:1,b12:1,b13:1,b14:1,b15:1;
```

```
};
```

```
union w16{    /* i、byte、bit共享存储 */
```

```
    short                i;
```

```
    struct w16_bytes     byte;
```

```
    struct w16_bits      bit;
```

```
};
```

字段结构与联合的应用

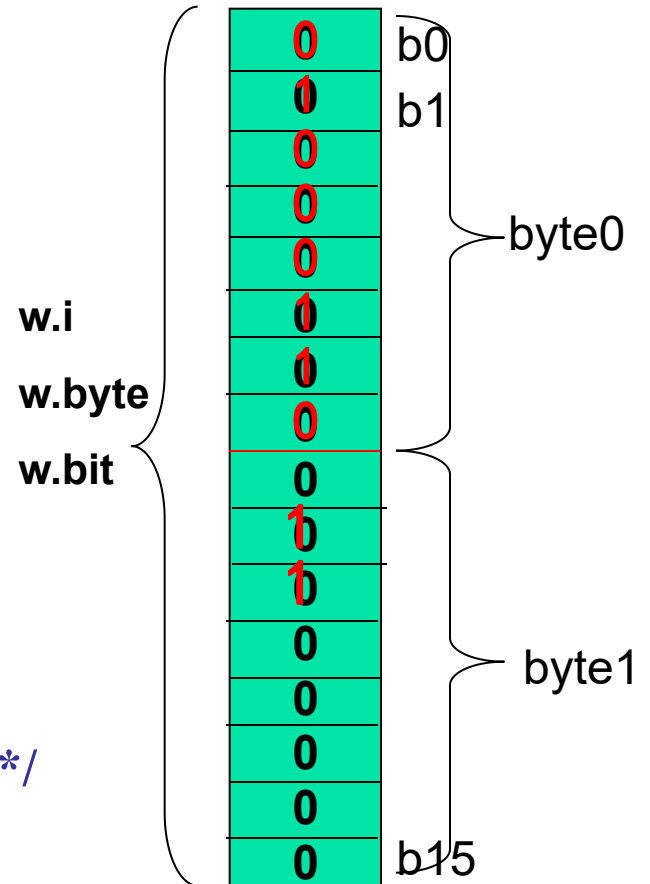
```
int main(void)
{
    union w16 w={0}; /* w.i为0 */

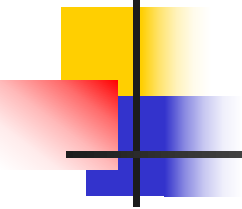
    w.bit.b9=1; /* 相当于byte1为2 */
    w.bit.b10=1; /* 相当于byte1为6 */

    w.byte.byte0=0x62;

    printf("w.i=0x%x\n",w.i); /* 按整型解释 */
    return 0;
}
```

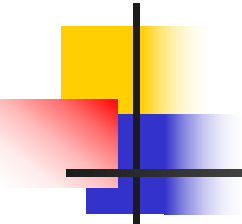
运行结果: **w.i=0x662**





9.8 结构指针的应用

- ◆ 单向链表的建立
- ◆ 单链表的删除
- ◆ 在链表中查找指定的元素
- ◆ 插入一个新元素到链表
- ◆ 删除链表中的一个元素
- ◆ 链表排序



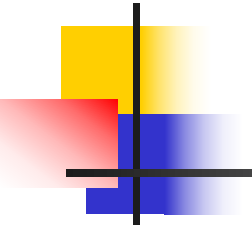
顺序存储和链式存储

- **顺序存储：数组**，是静态数据结构，其存储是在变量声明时建立的，并且在程序的执行过程中存储大小不能改变。

----- **内存固定，连续**

- **链式存储：单向链表、双向链表、十字交叉链表**，是动态数据结构，其存储是在程序运行过程中通过调用系统提供的动态存储分配函数，向系统申请而逐步建立起来的。在程序运行过程中，所占存储的大小可以根据需要调节，使用完毕时可以通过释放操作将所获得的存储交还给系统供再次分配。

----- **内存可变，不连续**



C 的动态存储分配函数

◆ 动态存储分配函数的原型声明在 **<stdlib.h>** 中

◆ C提供下列与动态存储分配相关的函数

```
void * malloc(size_t size);
```

```
void * calloc(size_t n, size_t size);
```

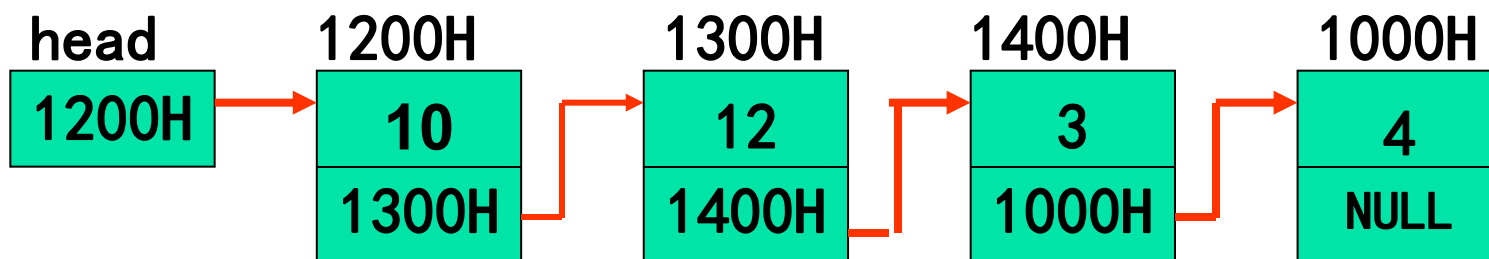
```
void * realloc(void *p_block, size_t size);
```

```
void free(void *p_block);
```

◆ 其中， **size_t** 表示 **unsigned int**。它是在 **<stdio.h>** 中通过 **typedef unsigned size_t;** 定义的。

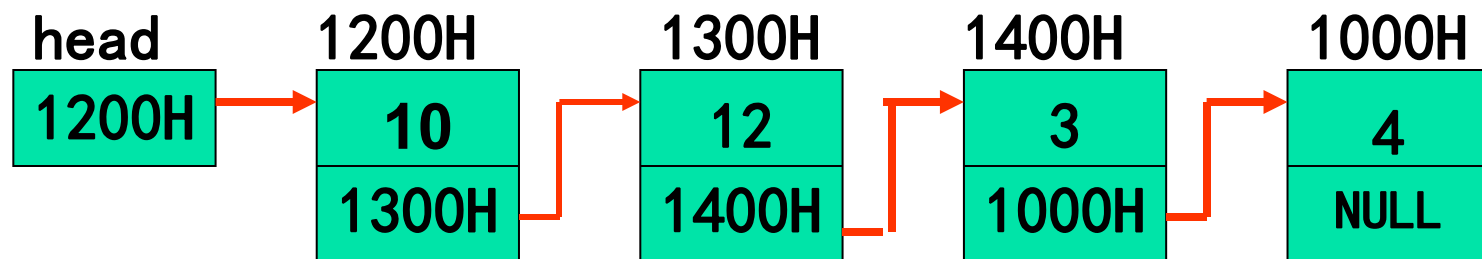
链 表

- 链表是一种常用的动态数据结构，它由一系列包含数据域和指针域的结点组成。
- 如果结点的指针域中只包含一个指向后一个结点指针，这种链表称为单向链表。



单向链表

单向链表



单向链表结构

head: 头指针，存放一个地址，指向第一个元素

结点: 链表中的元素，包括 **数据域**，用户数据
指针域，放下一个结点的地址

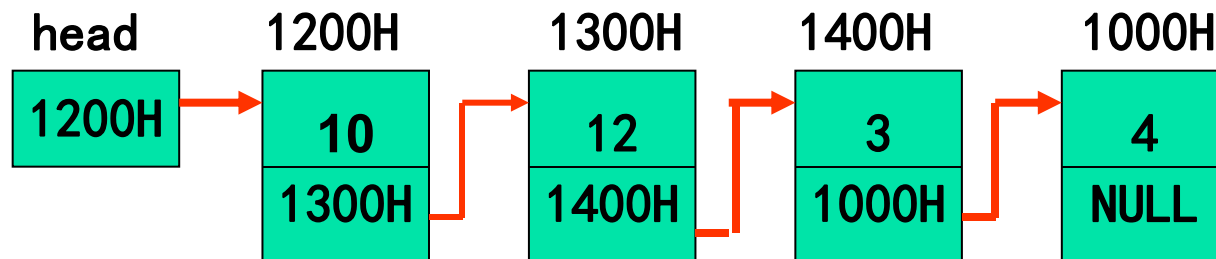
链尾: 最后一个结点，指针域为NULL

链表结点的结构类型定义

```
struct intNode {  
    int data;  
    struct intNode *next; /*指向该结构自身的指针*/  
};
```

头指针说明

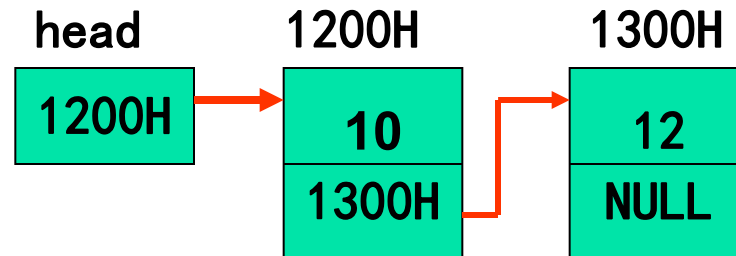
```
struct intNode *head;
```

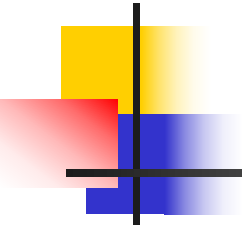


动态创建结点

- 可以通过**malloc**函数来动态创建结点。
- 如：

```
head=(struct intNode *)malloc(sizeof(struct intNode)) ;  
head->data=10;  
head->next=(struct intNode *)malloc(sizeof(struct intNode));  
head->next->data=12;  
head->next->next=NULL;
```





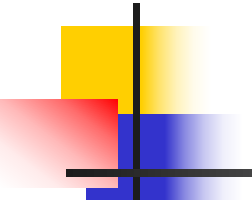
单向链表的建立

先进后出链（用链表实现栈）：

结点的排列顺序和数的输入顺序相反

先进先出链（用链表实现队列）：

结点的排列顺序和数的输入顺序相同



建立先进先出链表

用循环方式**建立先进先出链表**的步骤：

(1) 声明头指针，尾指针。

```
struct intNode *head=NULL,*tail;
```

(2) 创建第一个结点。包括：

① 给第一个结点动态分配存储并使头指针指向它。

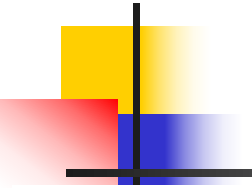
```
head=(struct intNode *)malloc(sizeof(struct intNode));
```

② 给第一个结点的数据域中成员赋值。

```
head->data=x;
```

③ 使尾指针也指向第一个结点。

```
tail=head;
```

建立先进先出链表

(3) 循环建立后续结点

如果没遇到结束标志，进行下列操作：

- ① 给后继结点动态分配存储并使前驱结点的指针指向它。

```
tail->next=(struct intNode *)malloc(sizeof(struct intNode));
```

- ② 使尾指针指向新建立的后继结点

```
tail=tail->next;
```

- ③ 给后继结点的数据域中成员赋值。

```
tail->data=x;
```

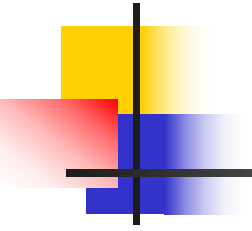
(4) 给尾结点（最后一个结点）的指针赋NULL值。

```
tail->next=NULL;
```

```

struct intNode *createList()  /* 用循环方式建立先进先出链表 */
{
    struct intNode *head=NULL,*tail=NULL;
    int x;
    scanf("%d",&x); /* 输入第1个整数 */
    if(x) {
        head=(struct intNode *)malloc(sizeof(struct intNode));
        head->data=x;    /* 对数据域赋值 */
        tail=head;      /* tail指向第一个结点 */
        scanf("%d",&x); /* 输入第2个整数 */
        while(x){      /* tail所指结点的指针域指向动态创建的结点 */
            tail->next=(struct intNode *)malloc(sizeof(struct intNode));
            tail=tail->next; /* tail指向新创建的结点 */
            tail->data=x; /* 向新创建的结点的数据域赋值 */
            scanf("%d",&x); /* 继续输入整数 */
        }
        tail->next=NULL; /* 对最后一个结点的指针域赋NULL值 */
    }
    return(head);
}

```



调用createList函数建立链表

```
struct intNode *createList(void);
```

```
int main(void)
```

```
{
```

```
    struct intNode* head=NULL;
```

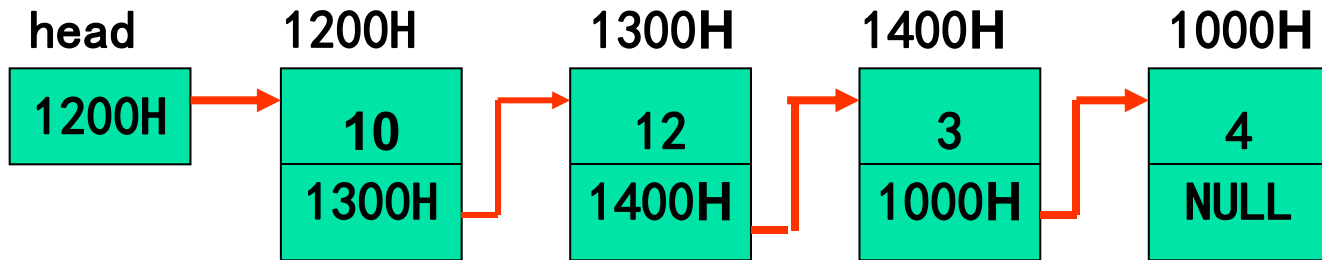
```
    head=createList();    /* 创建链表 */
```

```
    printList(head);    /* 输出链表结点中的数据 */
```

```
    return 0;
```

```
}
```

遍历链表



```
void printList( struct intNode *head)
{
    struct intNode *p=head; /*遍历指针p指向链头 */
    while(p!=NULL){ /* p非空 */
        printf("%d\t",p->data); //输出结点数据域中成员的值
        p=p->next; /*遍历指针p指向下一结点 */
    }
}
```

递归建立先进先出链表

```
struct intNode *createList()
```

```
{  
    struct intNode head=NULL;
```

```
    int x;
```

```
    scanf("%d",&x);
```

```
    if(x==0) /* 遇到结束标记，返回NULL */
```

```
        return NULL;
```

```
    else {
```

```
        head=(struct intNode *)malloc(sizeof(struct intNode));
```

```
        /* head指向新创建的结点 */
```

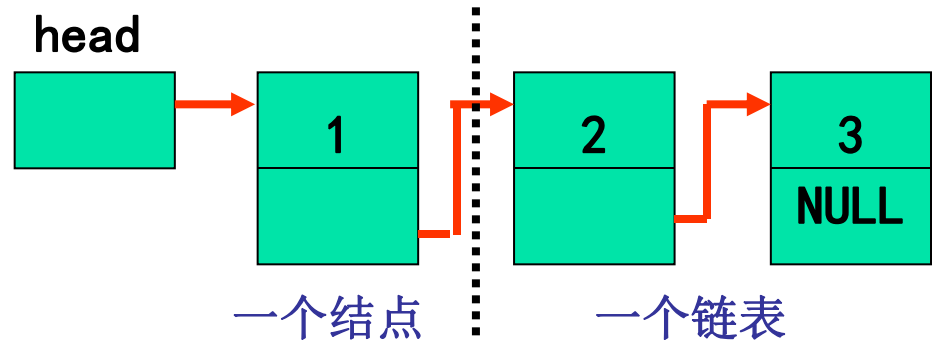
```
        head->data=x;          /* 对新创建结点的数据域赋值 */
```

```
        head->next=createList(); /* 递归创建下一结点 */
```

```
        return head;          /* 返回链头地址 */
```

```
    }
```

```
}
```



统计链表中结点的数目

/* 循环遍历法统计链表head中结点的数目 */

int **countNodes**(struct intNode *head)

{

struct intNode *p=head;

int num=0;

while (**p!=NULL**){

num++;

p=p->next;

}

return num;

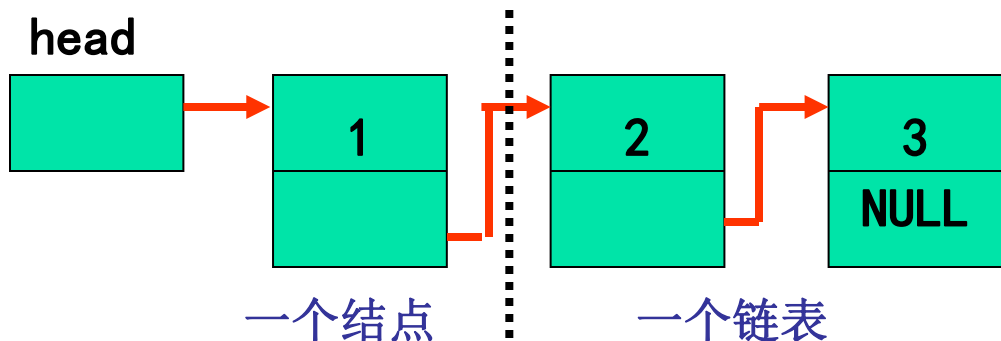
}

P!=0

P

统计链表中结点的数目

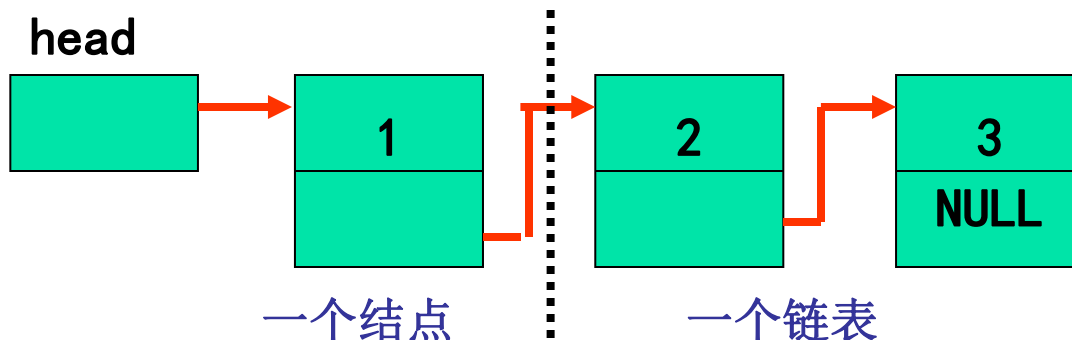
```
/* 用递归法统计链表head中结点的数目 */
int countNodes_recursive(struct intNode *head)
{
    struct intNode *p=head;
    if(p)
        return (1+countNodes_recursive(p->next));
    else
        return 0;
}
```



查找结点

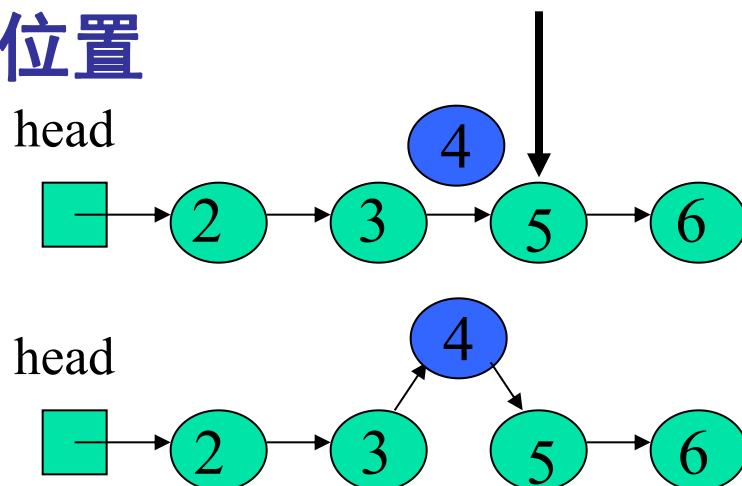
/* 用递归法查找**链表**中数据成员值与形参**n**相同的结点。找到，返回该结点的地址，否则，返回NULL。 */

```
struct intNode *findNodes(struct intNode *head, int n)
{
    struct intNode *p=head;
    if(p) { /* 链表非空，查找 */
        if(p->data==n) return p; /* 找到，返回该结点的地址 */
        else
            return (findNodes(p->next,n)); /* 递归查找 */
    }
    else return NULL;
}
```



插入结点

(1) 寻找插入位置



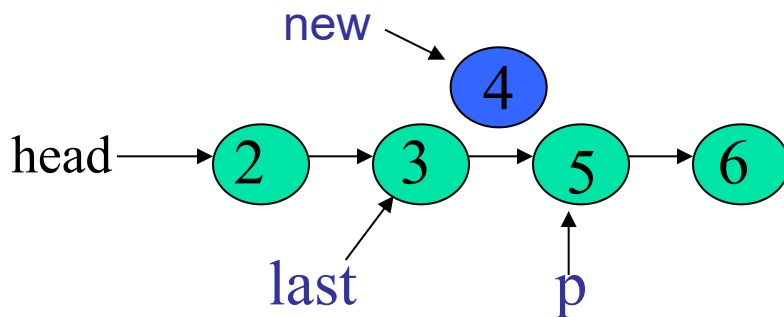
插入点的位置可能是链头、链尾、或者链中。
插入方式有将结点作为插入点的新后继结点和
插入点的新前驱结点两种。
下面介绍将结点作为插入点的新前驱结点。

插入结点

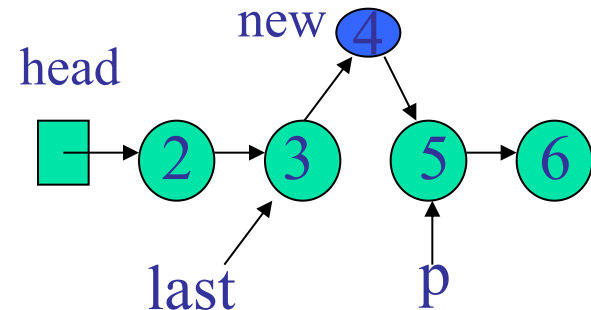
(2) 将新结点插入链表

插入点是链中 (p 和 $last$ 之间)

需要两个遍历指针，一个是指向插入点的遍历指针 p ，另一个是指向插入点前驱结点的指针 $last$ 。设新结点由 new 指针所指，它将插入在 p 和 $last$ 所指结点之间。



插入前结点间的指向关系



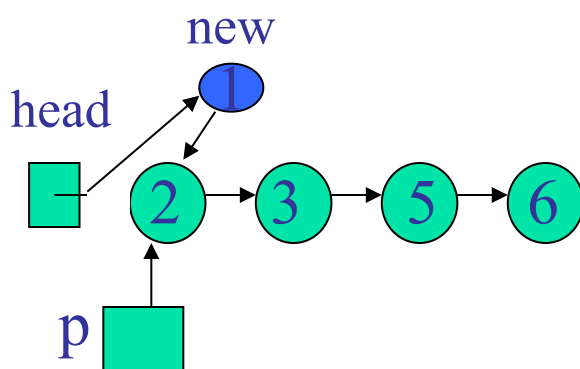
插入后结点间的指向关系

$last \rightarrow next = new;$

$new \rightarrow next = p;$

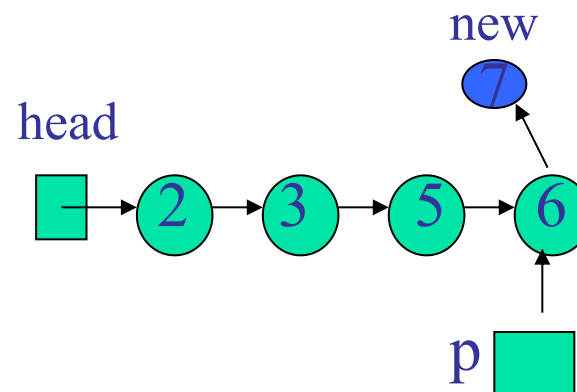
插入结点

插入点是链头



head=new;
new->next=p;

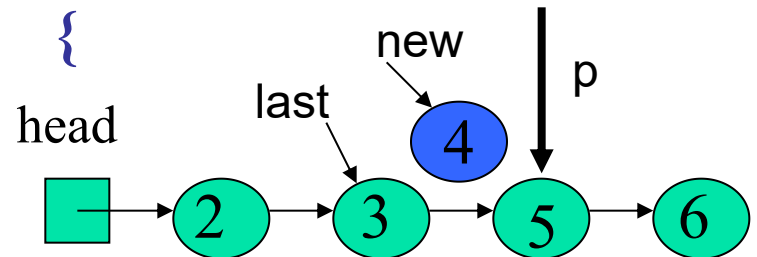
插入点是链尾



new->next=NULL;
p->next=new;

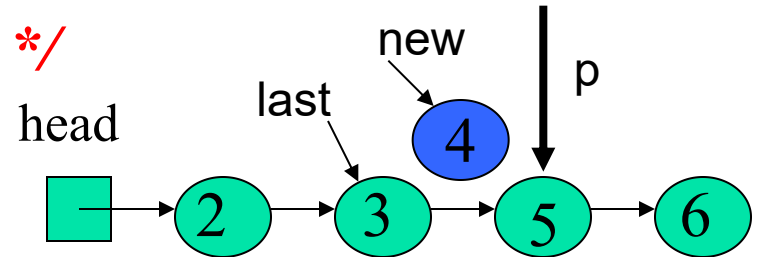
插入结点

```
/* 在一个升序链表中插入值为x的结点，返回新结点的地址。 */  
#define LEN sizeof(struct intNode)  
struct intNode *insertNode ( struct intNode **hp, int x )  
{  
    struct intNode *p, *last, *new;  
    new = (struct intNode *)malloc(LEN); /* 创建一个新节点 */  
    new->data=x;  
    p=*hp;  
    /* 寻找插入位置 */  
    while( p!=NULL && x > p->data ) {  
        last=p;  
        p=p->next;  
    }
```



插入结点

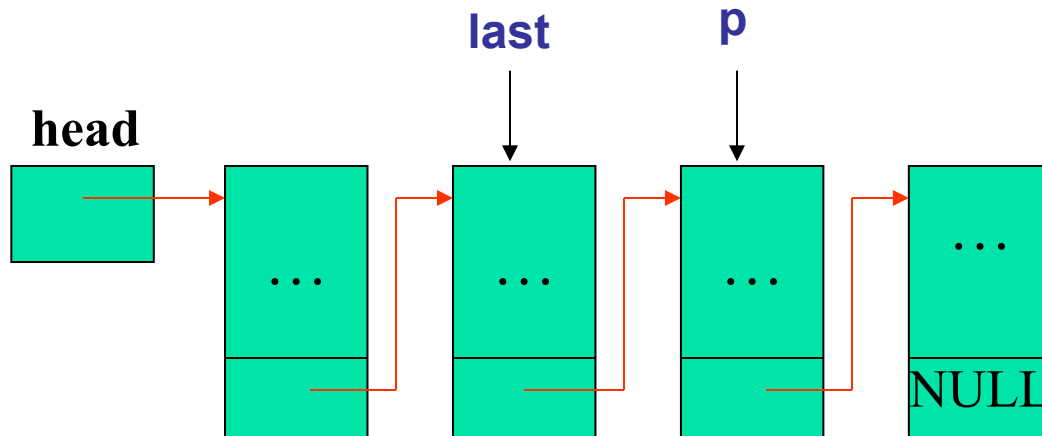
```
if ( p == *hp ) { /* 插入点是链头 */
    *hp=new; /* 新节点为链头 */
    new->next=p;
}
else if (p==NULL) { /* 插入点是链尾 */
    new->next=NULL; /* 新节点为链尾 */
    last->next=new;
}
else { /* 插入点是链中 */
    new->next=p; /* 新节点为 p 的前驱结点 */
    last->next=new;
}
return new;
}
```



删除结点

(1) 查找被删结点

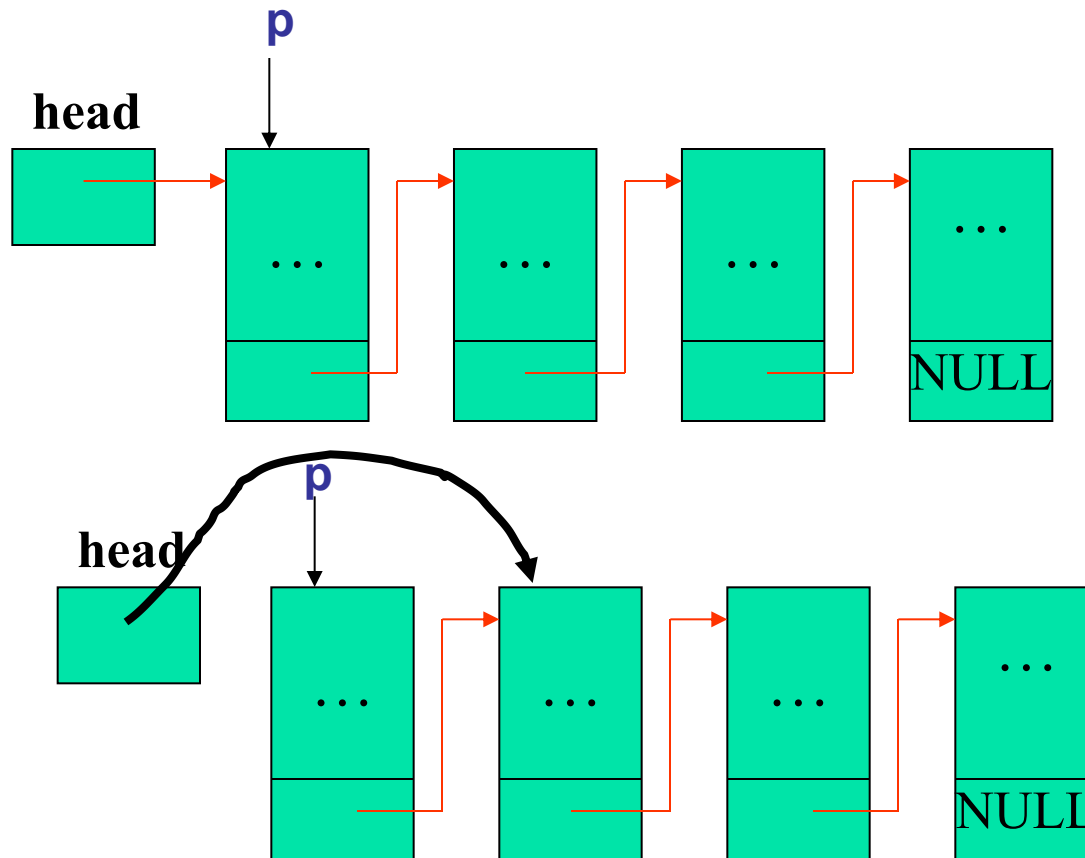
需要指向被删除结点的遍历指针 p 和指向被删除结点的前驱结点的遍历指针 $last$ 。



删除结点

(2) 删除p所指结点，即改变连接关系

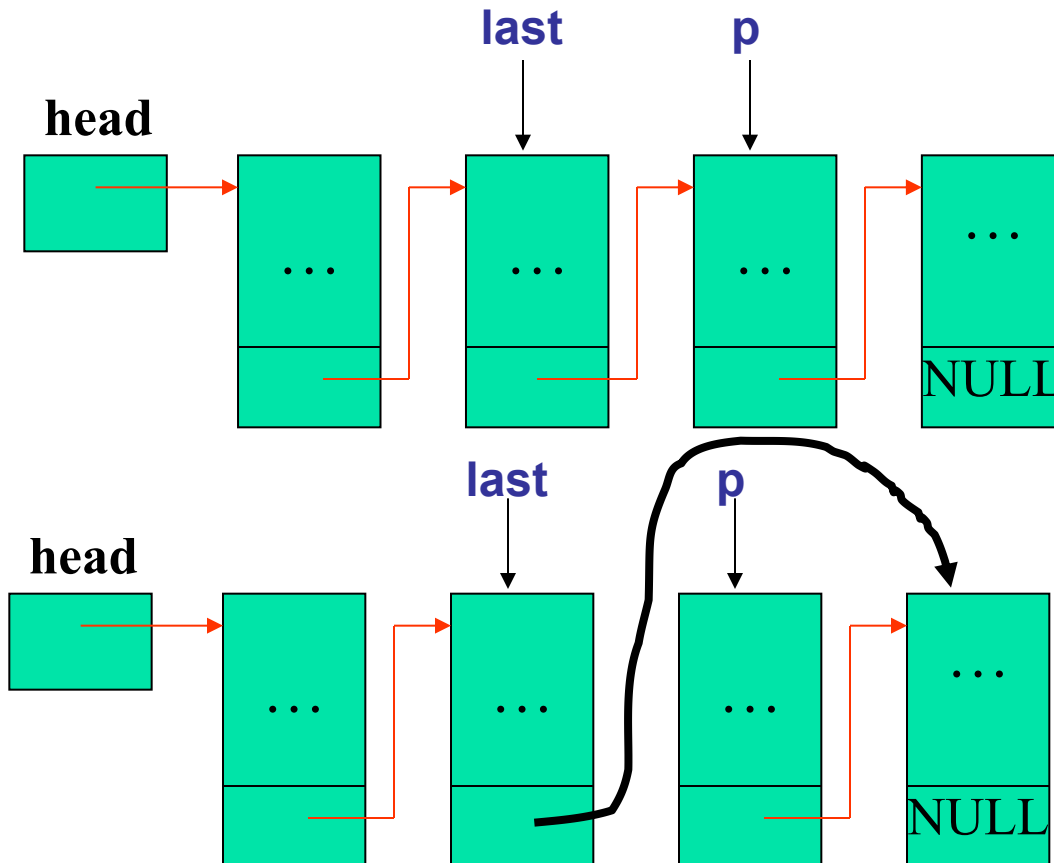
链头： 修改 *head*: $head = p \rightarrow next$

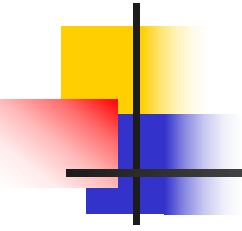


删除结点

非链头: 修改前驱结点的指针域:

$last \rightarrow next = p \rightarrow next;$





删除结点

(3) 释放 *p* 所指的存储区

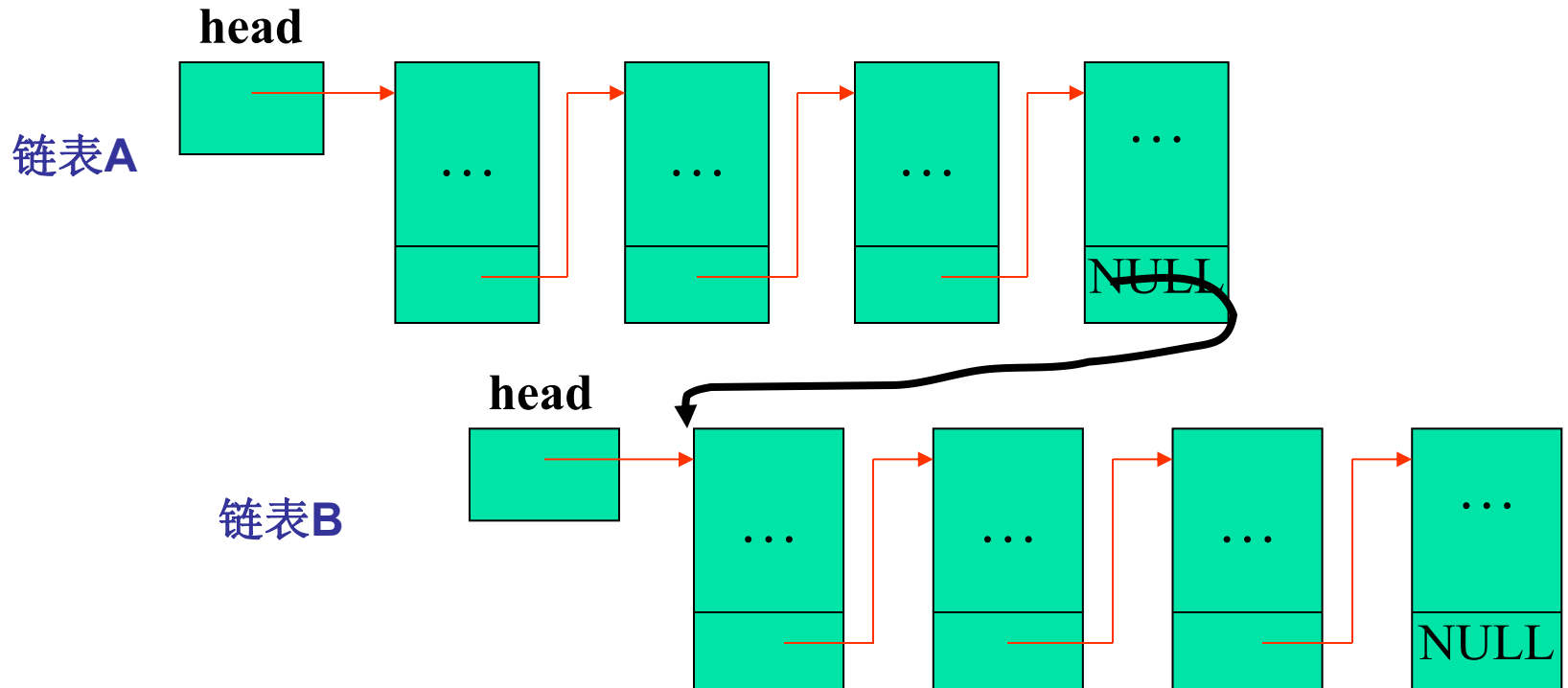
`free(p);`

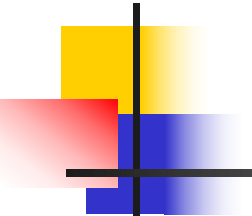
删除结点

```
/* 删除值为n的结点，成功删除返回1；否则，返回0。 */
int deleteNodes(struct intNode **hp,int n)
{
    struct intNode *p,*last;
    p=*hp;
    while(p !=NULL&& p ->data!=n){ /* 查找成员值与n相等的结点 */
        last = p;          /* last指向当前结点 */
        p = p ->next;      /* p指向下一结点 */
    }
    if(p ==NULL) return 0 ; /* 没有符合条件的结点 */
    if(p ==*hp) /* 被删结点是链头 */
        *hp= p ->next;
    else /* 被删结点不是链头 */
        last->next= p ->next;
    free(p); /* 释放被删结点的存储 */
    return 1;
}
```

归并链表

对于非空链表A、B,将链表B归并到链表A, 指将链表A的链尾指向链表B的链头所形成的一个新的链表。





归并链表

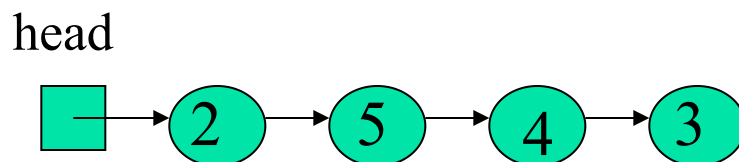
(1) 遍历链表A找到其链尾

(2) 将链表B的头指针值赋给链表A链尾的指针域

```
void conLists ( struct intNode *a, struct intNode *b )  
{  
    struct intNode *p=a;  
    while(p-> next != NULL ) {  
        p=p->next;  
    }  
    p->next=b;  
}
```

链表排序

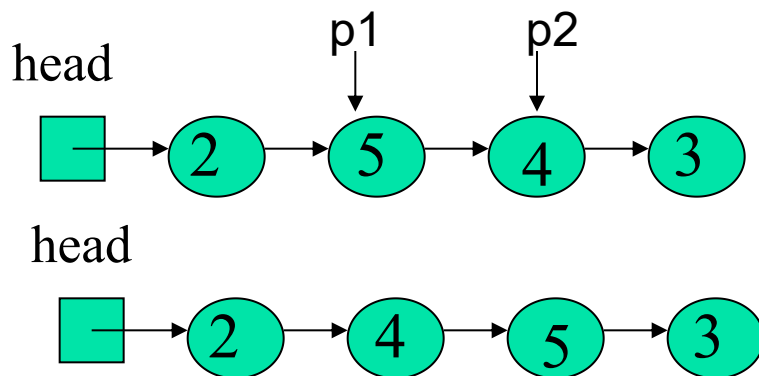
- 链表排序是指将链表的结点按某个数据域从小到大（升序）或从大到小（降序）的顺序连接。



链表排序法1

排序中对链表结点的交换有**两种方法**,

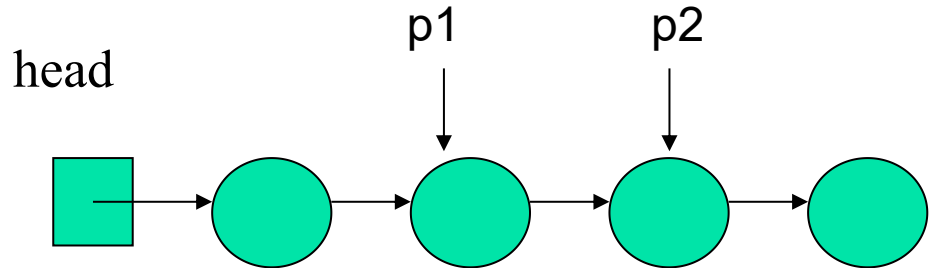
(1) 交换结点的数据域, 不改变结点间的指向关系。



```
if(p1->data > p2->data){  
    int t;  
    t=p1->data; /* 交换数据域 */  
    p1->data=p2->data;  
    p2->data=t;  
}
```

链表排序法1

```
struct std {  
    char num[12];  
    char name[9];  
    int score;  
};  
struct list {  
    struct std data;  
    struct list *next;  
};
```



```
if(p1->data->score > p2->data->score)  
{  
    struct std t;  
    t=p1->data; /* 交换数据域 */  
    p1->data =p2->data;  
    p2->data =t;  
}
```

链表排序函数

```
void sortList(struct intNode *head)
```

```
{
```

```
    struct intNode *p1,*p2;
```

```
    int t;
```

```
    for(p1=head;p1!=NULL;p1=p1->next)
```

```
        for(p2=p1->next;p2!=NULL;p2=p2->next)
```

```
            if(p1->data > p2->data){
```

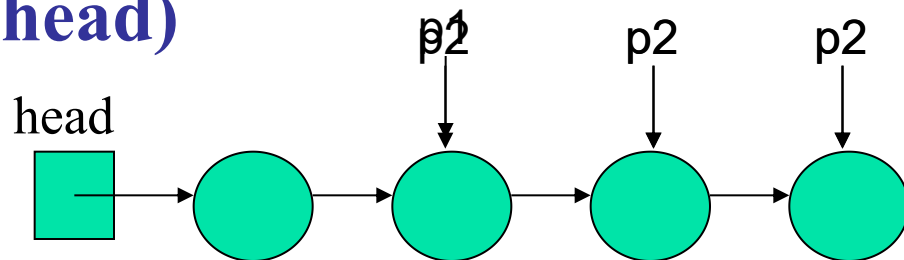
```
                t=p1->data;
```

```
                p1->data=p2->data;
```

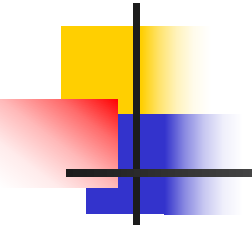
```
                p2->data=t;
```

```
            }
```

```
}
```



/* 交换数据域 */



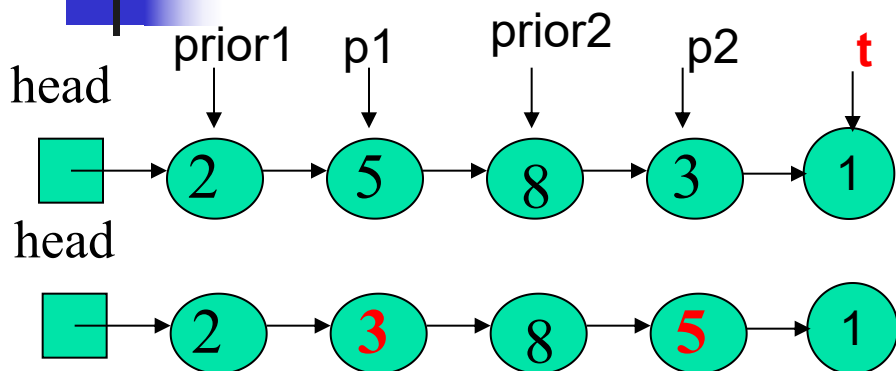
链表排序法2

(2) 改变结点的连接，操作较为复杂。

在数据域较为简单的情况下，多采用交换结点的数据域的方法进行链表排序。

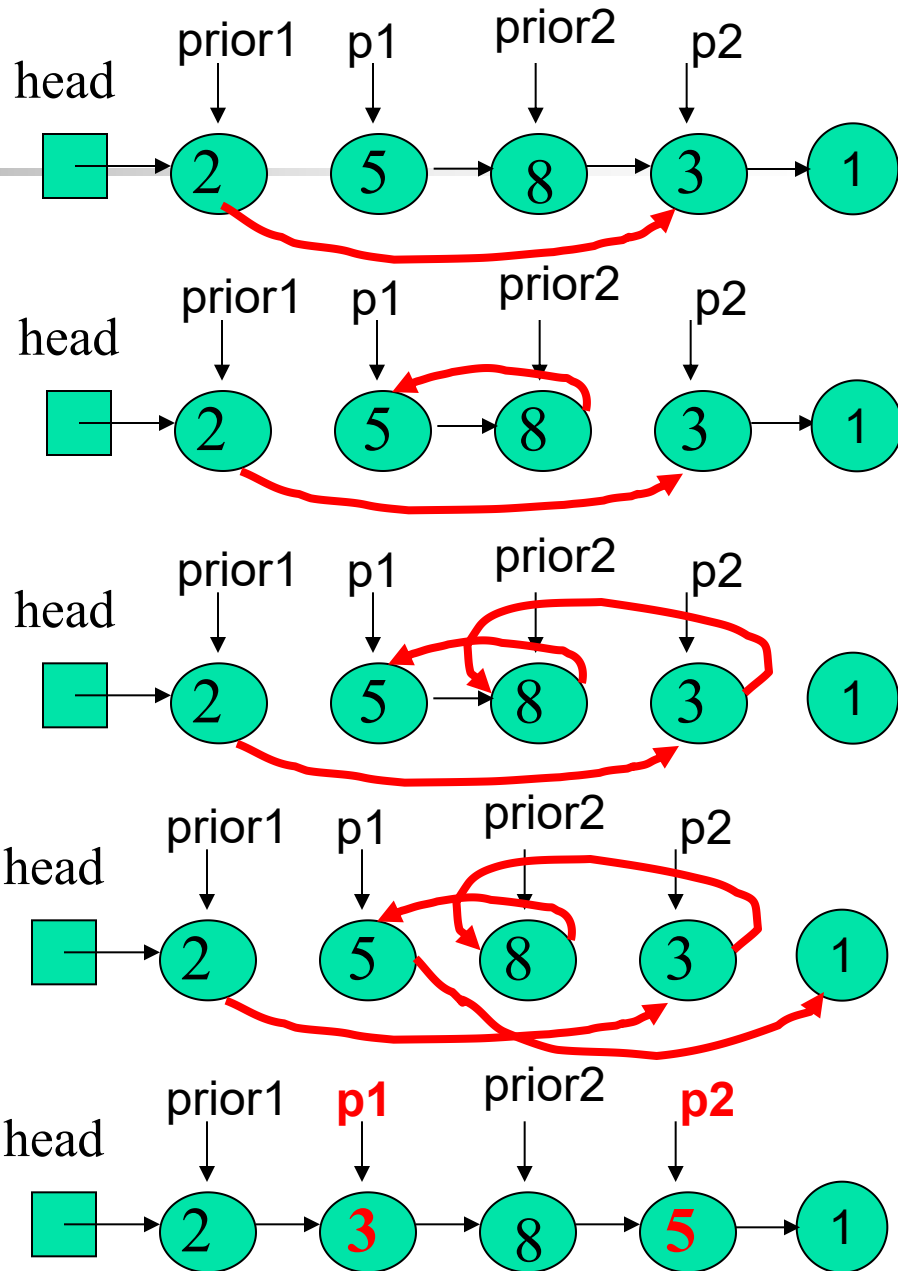
在数据域较为复杂，成员较多，采用改变链表结点之间的连接实现结点的交换的排序方法则效率较高。

交换p1和p2

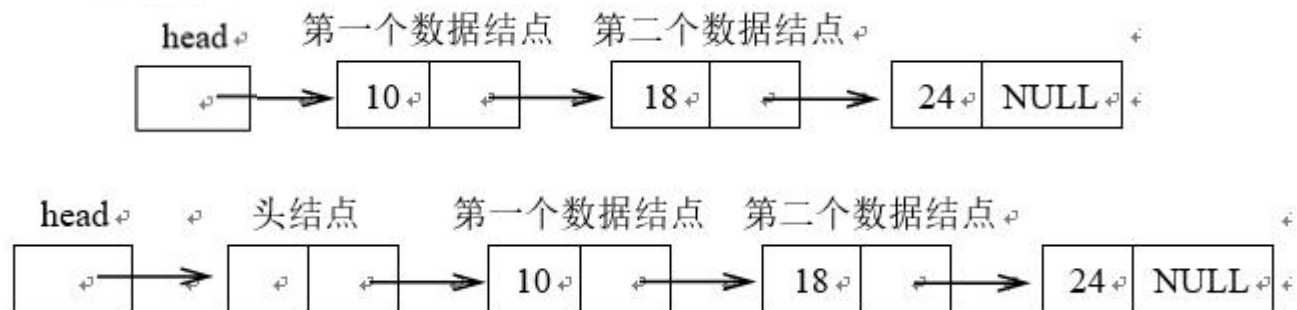


```

if(p1->data>p2->data){
    t=p2->next;
    prior1->next=p2;
    prior2->next=p1;
    p2->next=p1->next;
    p1->next=t;
    p2=p1;
    p1=prior1->next;
}
    
```



新增头结点



// 新增头指针结点的方法

```
void sortList (struct intNode **hp)
```

```
{
```

```
    struct intNode *prior1,*prior2,*p1,*p2,*t;
```

```
    int i=0;
```

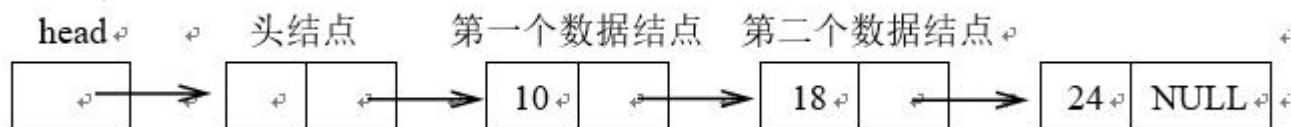
```
    p1=(struct intNode *)malloc(sizeof(struct intNode)); // 新增结点
```

```
    p1->next=*hp;      // 使新结点成为头结点
```

```
    (*hp)=prior1=p1;
```

新增头结点

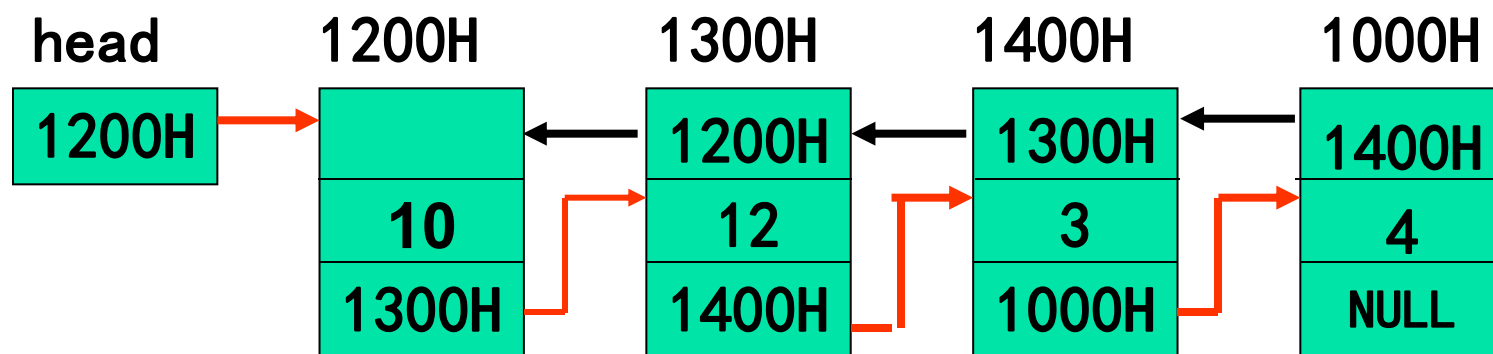
```
for(p1=prior1->next;p1->next!=NULL;prior1=p1,p1=p1->next)
    for(p2=p1->next,prior2=p1;p2!=NULL;prior2=p2,p2=p2->next)
        if(p1->data>p2->data){
            t=p2->next;
            prior1->next=p2;
            prior2->next=p1;
            p2->next=p1->next;
            p1->next=t;
            p2=p1;
            p1=prior1->next;
        }
```



```
p1>(*hp);          /* p1指向新增头指针结点 */
(*hp)=(*hp)->next; /* (*hp)指向排序后链表的链头 */
free(p1);          /* 释放新增头指针结点 */
}
```

双向链表

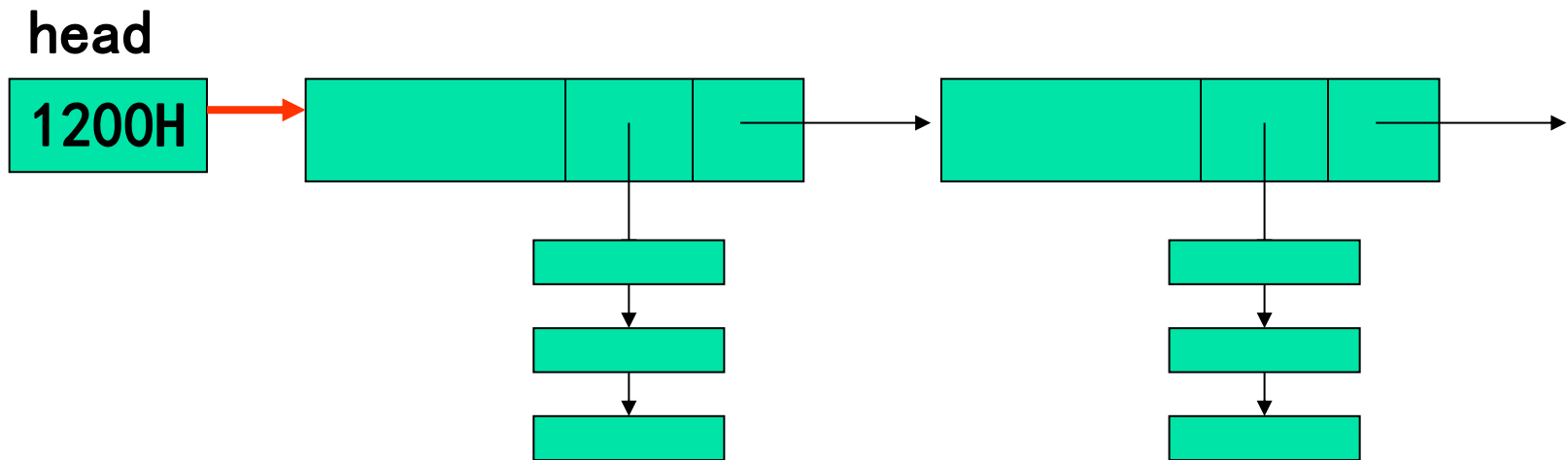
- 如果结点的指针域包含两个指针，且一个指向前一个结点，另一个指向后一个结点，这种链表称为**双向链表**。



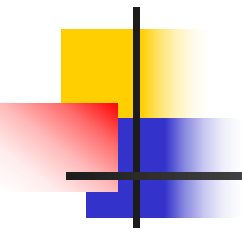
双向链表结构

十字交叉链表

- 如果结点的指针域包含两个指针，且一个指向后一个结点，另一个指向另外一个链表，这种链表称为**十字交叉链表**。



十字交叉链表结构



用十字交叉链表保存学生基本信息和成绩

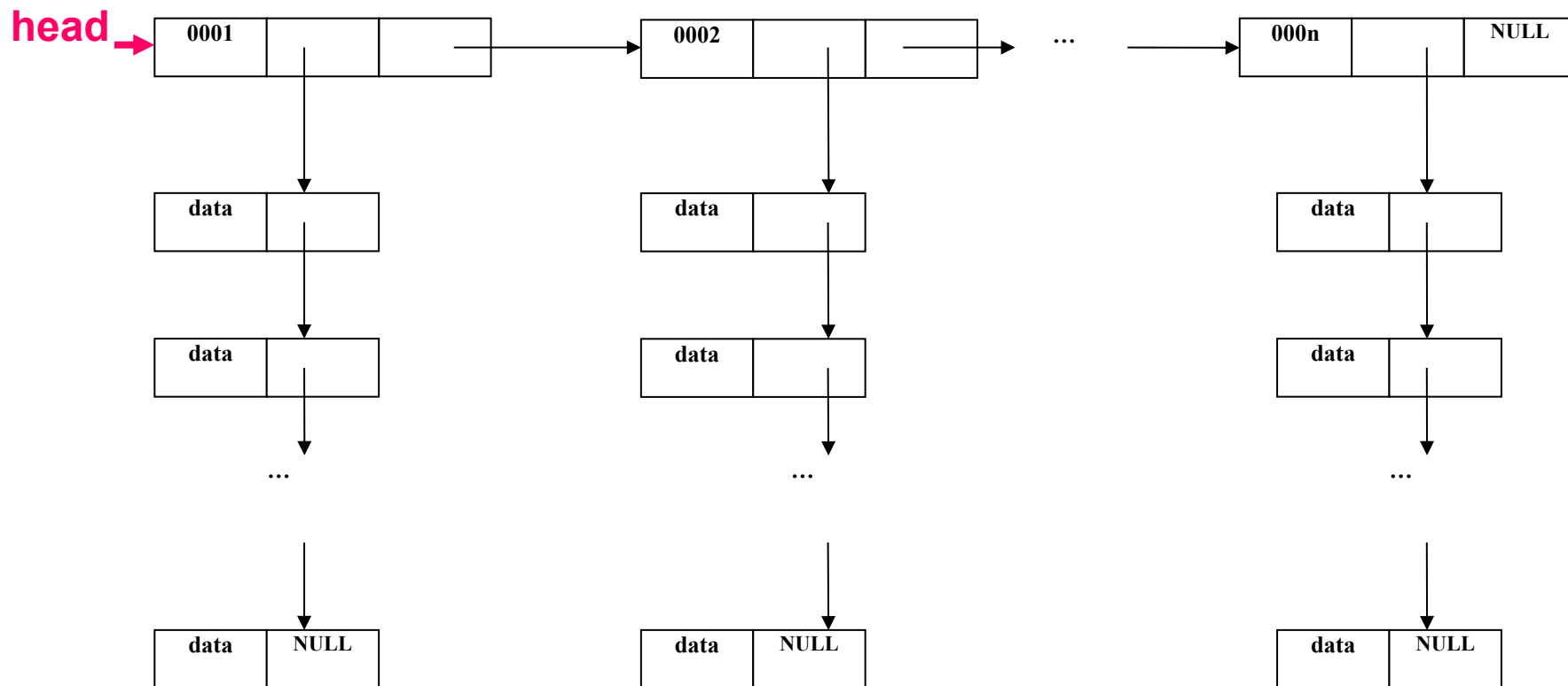
学生基本信息表

学号	姓名	性别	年龄	家庭住址	联系电话	备注
0001	aaa	m	18	hubei, wuhan	12345678	Abc def
0001	bbb	f	18	hunan, changsha	76545678	Aaaaaa bbb
...						
00xx	zzz	m	19	hubei, honghu	32145678	Nnn kkk

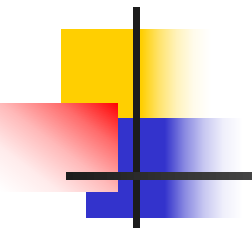
学生学习成绩表

学号	高等数学	普通物理	电工基础	演讲与口才	欧洲文化	论语初探
0001	86	85	73	×	80	×
0002	77	83	76	82	87	75
0003	87	82	81	×	×	86
...						
00xx	89	87	85	×	×	×

十字交叉链表



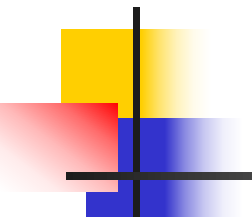
水平方向:学生基本信息链; 垂直方向:各学生课程成绩链



学生课程成绩结点的结构类型定义

该类型的结构变量可以构成纵向单向链表的结点。

```
typedef struct score_tab{  
    char    num[5];          /* 学号 */  
    char    course[20];      /* 课程名称 */  
    int     score;           /* 成绩 */  
    struct  score_tab *next; /* 指向下一个课程成绩结点 */  
} courses;
```



学生基本信息结点的结构类型定义

该类型的结构变量可以构成十字交叉链表中横向学生基本信息链的结点。

```
typedef struct student_tab{  
    char    num[5];           /* 学号 */  
    char    name[10];        /* 姓名 */  
    char    sex;              /* 性别 */  
    int     age;              /* 年龄 */  
    char    addr[30];         /* 家庭住址 */  
    char    phone[12];        /* 联系电话 */  
    char*    memo;            /* 备注字段 */  
    courses *head_score;    /* 指向成绩链的头指针 */  
    struct student_tab *next; /* 指向下一个结点 */  
} studs;
```